CS413 · MIDTERM 1 PROJECT

Compiling Extended LAMBDA into Pure λ-Calculus

A compositional compiler, a shared call-by-value evaluator, and bounded decoders, in Python 3.12

```
C[2 + 3 * 4] → (((lam m. (lam n. (lam s. (lam z. ((m s) ((n s) z)))))) …
```

Filip Jasionek · October 25, 2026

THE TASK

Translate the program, never its answer

Source AST

```
D0Eop2("+", D0Eint(2),
  D0Eop2("*", D0Eint(3), D0Eint(4)))
```



Pure target term (194 characters)

```
(((lam m. (lam n. (lam s. (lam z. ((m s) ((n
s) z)))))) (lam s. (lam z. (s (s z))))
(((lam m. (lam n. (lam s. (m (n s))))) …
```

Three forms only

Output holds only Var, Lam and App. No numbers, no named primitives.

β-reduction does the math

Python builds literal numerals and counts the decoded result. Nothing else.

Compile, don’t run

The compiler never calls the evaluator, so even a diverging program compiles.

SOURCE LANGUAGE

Extended LAMBDA: the Assignment 5 AST

Group	Constructors	Meaning
Literals	D0Eint(n), D0Ebtf(b)	Naturals n ≥ 0; true, false
Functions	D0Evar, D0Elam, D0Eapp	One-parameter lambdas, application
Operators	D0Eop1("not"), D0Eop2(op)	+ – * (truncated), < <= > >= == !=, and, or
Control	D0Eif0, D0Elet	Boolean if; non-recursive let
Data	D0Epair, D0Epfst, D0Epsnd	Ordered pairs and projections
Recursion	D0Efix(f, x, body)	Recursive function f with parameter x

Call-by-value, left to right. if, and, or evaluate only what they select.

Rejected at compile time: negative literals and other operators, even in dead code.

TARGET LANGUAGE

Pure λ-calculus with a separate three-node AST

```
Term ::= Var(name)
       | Lam(name, Term)
       | App(Term, Term)
```

The one rule: a beta-value step

`(lam x. body) v → body[x := v]`

v must already be a lambda

Printer: x, (lam x. body), (fn arg)

Weak, left-to-right CBV: function, then argument, then substitute. Never under a lambda, no eta.

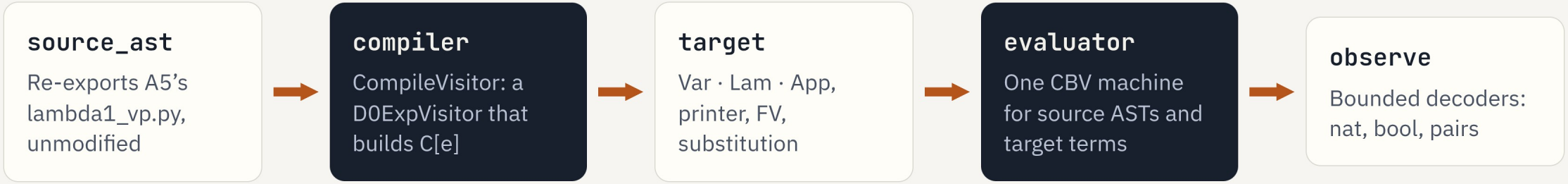
Capture-avoiding substitution: renames a binder only if it would capture.

Frozen dataclasses: input terms are never mutated.

Alpha-equivalence via De Bruijn indices, ported from Assignment 5.

ARCHITECTURE

Five modules on the main path, three alongside



church
Closed combinators the compiler splices in

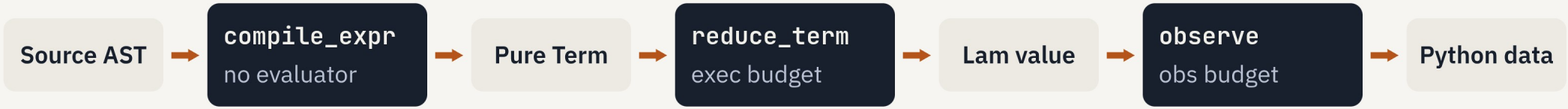
source_visitors
Printer, substitution, FreshNames, from A5

pipeline · demo
Compile → execute → observe, separate budgets

Import boundaries are tested: the compiler never imports the evaluator, and the evaluator never imports the compiler.

DESIGN PATTERN · PIPELINE

Pipes and filters: three stages, one wiring point



Stages share no state

Each takes an immutable input and returns a new output, so each is tested on its own.

Rules become structure

Compile can’t evaluate: it never imports the evaluator. Separate stages give separate budgets.

One wiring point

Only pipeline.py connects the stages. The CLI demo and the web extension both reuse it.

Running the source evaluator on the original AST is a **separate fourth path**: an oracle the pipeline is checked against.

DESIGN PATTERN · VISITOR

Visitor: new operations on an AST we can’t change

```
class CompileVisitor(D0ExpVisitor[Term]):

  def visit_let(self, dexp):
    init = dexp.arg2.accept(self)
    body = dexp.arg3.accept(self)
    return App(Lam(dexp.arg1, body), init)

  def visit_if0(self, dexp):
    return select(dexp.arg1.accept(self),
      dexp.arg2.accept(self), dexp.arg3.accept(self))
```

node.accept(v) calls v.visit_let(node)

Open for extension, closed to change

lambda1_vp.py stays untouched. Compile, print, substitute and FV are each a new visitor class.

One method = one rule

13 visit methods, one per constructor, each its translation rule. Each recurses only on its children: compositional by construction.

Completeness is enforced

All 13 methods are abstract. A visitor with a missing case can’t be instantiated.

DESIGN PATTERNS

Supporting patterns, each with a job

Pattern	Where	Why
Composite	Both ASTs	Terms contain terms; every operation recurses
Abstract machine (CK-style)	evaluator.py	Frame stack: no recursion limit, residual on exhaustion
Combinator library	church.py	Closed building blocks that can’t capture names
Smart constructors	num(n), lam, app	Readable term building; num rejects n < 0
Immutable value objects	Frozen dataclasses	No mutation; safe sharing; cached free variables
Result objects	Reduction, Observation	Exhaustion is data; exceptions only for real errors
Table-driven dispatch	BINARY, UNARY	One place maps each operator to its combinator
Facade, Adapter	source_ast.py, web tools.py	One import point for the AST; HTTP to core

Not a factory system. The only real factory is Flask’s create_app, in the optional web extension.

ENGINEERING PRINCIPLES

Six principles, each checked by code or tests

Separation of concerns

One job per module: compile, encode, reduce, decode, present.

Acyclic dependencies

Compiler and evaluator never import each other; the core never imports extensions. Tested.

Immutability

Frozen nodes. Tests snapshot inputs and show nothing is mutated.

Determinism

Fresh names depend only on the term. Same input, same output, same step count.

Don’t repeat yourself

One select() for if, and, or. One evaluator, one pipeline, reused A5 code.

Fail fast

Bad input is rejected at compile time, even in dead code. Bad budgets and open programs are refused.

KISS and YAGNI: a plain substitution reducer matches the spec’s definition, and the parser and web UI stay out of the core.

PROCESS

Small increments, with an AI pair programmer

1 Reuse A5 code · 2 Target AST · 3 Reducer · 4 Source path · 5 Church library · 6 Compiler · 7 Demo and web · 8 Scope and purity checks · 9 Docs and deck

Pair programming, driver and navigator

I navigated: picked each milestone, set constraints (“copy it, don’t import”) and challenged designs (“why use a stack?”, “why reject eta?”). Claude drove, writing code and tests. I reviewed by reading and running them.

Agile practices, not full Scrum

Working, tested software after every increment. REQUIREMENTS.md as the backlog, its IDs (ACC-10…) cited in tests. Course corrections when review found a problem.

CHURCH ENCODINGS

Data is a function that chooses or repeats

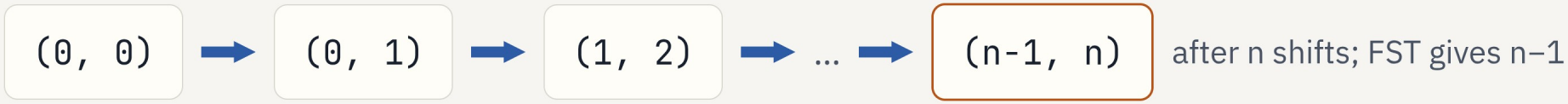
Name	Pure term (church.py)
TRUE, FALSE	lam t. lam f. t · lam t. lam f. f
NUM(n)	lam s. lam z. s (s … (s z)) — s applied n times
PAIR	lam a. lam b. lam p. p a b
FST, SND	lam p. p TRUE · lam p. p FALSE
SUCC	lam n. lam s. lam z. s (n s z)
ADD	lam m. lam n. lam s. lam z. m s (n s z)
MUL	lam m. lam n. lam s. m (n s)
NOT	lam b. b FALSE TRUE

Every combinator is **closed** and expanded in place. NUM(0) and FALSE are the same term.

PREDECESSOR

PRED slides a two-slot window up the numeral

```
SHIFT = lam p. PAIR (SND p) (SUCC (SND p))
PRED = lam n. FST (n SHIFT (PAIR NUM(0) NUM(0)))
```



after n shifts; FST gives n−1

PRED 0 = 0

Zero shifts leave (0, 0), so FST returns 0. No special case.

SUB = lam m. lam n. n PRED m

Apply PRED n times to m. Truncation (2 − 5 = 0) falls out of PRED 0 = 0.

COMPARISONS

Every comparison reduces to “is m − n zero?”

```
ISZERO = lam n. n (lam x. FALSE) TRUE

LEQ m n = ISZERO (SUB m n)
GEQ m n = LEQ n m
LT  m n = NOT (LEQ n m)
GT  m n = NOT (LEQ m n)
EQ  m n = AND (LEQ m n) (LEQ n m)
NEQ m n = NOT (EQ m n)
```

Why it works

Truncated m − n is 0 exactly when m ≤ n. The other five follow by swapping or negating.

Strict AND is fine here

Both LEQ results are already values. Source and/or compile to thunks instead.

TRANSLATION RULES

One rule per constructor, combined from the children

`C[n]`	`NUM(n)` (n ≥ 0, else CompileError)
`C[true], C[false]`	`TRUE, FALSE`
`C[x]`	`x`
`C[lam x. e]`	`lam x. C[e]`
`C[e1 e2]`	`C[e1] C[e2]`
`C[not e]`	`NOT C[e]`
`C[e1 op e2]`	`OP C[e1] C[e2]` OP: ADD SUB MUL LT LEQ GT GEQ EQ NEQ
`C[let x = e1 in e2]`	`(lam x. C[e2]) C[e1]`
`C[pair e1 e2]`	`PAIR C[e1] C[e2]`
`C[fst e], C[snd e]`	`FST C[e], SND C[e]`
`C[if c then t else f]`	`(C[c] (lam u. C[t]) (lam v. C[f])) I`
`C[e1 and e2]`	`(C[e1] (lam u. C[e2]) (lam v. FALSE)) I`
`C[e1 or e2]`	`(C[e1] (lam u. TRUE) (lam v. C[e2])) I`
`C[fix f(x). e]`	`Z (lam f. lam x. C[e])`

I = lam w. w. Thunk parameters u, v are chosen fresh. Highlighted rules delay evaluation.

DELAYED BRANCHES

Thunks stop call-by-value from running both branches

`if true then 1 else LOOP where LOOP = (fix loop(x). loop x) 0`

Naive: pass the branches

```
TRUE NUM(1) C[LOOP]
```

C[LOOP] is an argument, so CBV evaluates it before TRUE can choose.

Never returns

Compiled: pass thunks, force one

```
(TRUE (lam u. NUM(1)) (lam v. C[LOOP])) I
→ (lam u. NUM(1)) I
→ NUM(1)
```

3 β-steps, decodes to 1

A lambda is already a value, so CBV leaves both thunks alone. **and** and **or** use the same select(): false and LOOP → false.

RECURSION

Z, the call-by-value fixed point, replaces Y

Y unfolds forever

```
Y = lam g. (lam x. g (x x))
           (lam x. g (x x))
```

The argument x x is an application, so CBV evaluates it before g runs, and that rebuilds the same term.

Diverges (tested)

Z waits for an argument

```
Z = lam g. (lam x. g (lam v. x x v))
           (lam x. g (lam v. x x v))
```

Eta-expanding to lam v. x x v makes the argument a value. Self-application waits for a call.

One unfolding per call

```
C[fix f(x). e] = Z (lam f. lam x. C[e])
```

fix f(f). f becomes lam f. lam f. f. The inner binder wins, so (fix f(f). f) 3 = 3.

NAMES AND SCOPE

Fresh names are chosen, never reserved

Only thunks add binders

The parameter is the first u_1, u_2, … (or v_k) that occurs nowhere in the body, free or bound. Deterministic, and it depends only on the body.

Combinators can’t capture

They are closed and only ever applied, never wrapped around user code. Their inner s, z, t, f stay private.

Branch mentions u_1

```
if c then u_1 else 0
→ ((c (lam u_2. u_1)) (lam v_1. …)) I
```

Source names like encoding names

```
let s = 2 in let z = 3 in s * z
→ 6 (MUL’s own s is untouched)
```

Result: FV(C[e]) = FV(e), checked on every test and demo program and on 250 random open ones.

SHARED EVALUATOR

One machine runs source ASTs and target terms

Explicit stack of context frames. Iterative, so 100,000-step runs never hit Python's recursion limit.

Shared cases: Var, Lam, App for both ASTs. Function, then argument, then one beta-value step.

Source-only frames: operators, and/or, if, let, pair, projections. Python arithmetic exists only here.

Target terms reach only the shared cases. No Church encoding is recognized. Open programs are rejected.

The budget contract

Term	Budget	Result
I	0	value, 0 steps
I I	0	exhausted, 0 steps
I I	1	value, 1 step
LOOP	100,000	exhausted, 100,000

Reduction(status, term, steps). Exhaustion returns the residual program and is not a proof of divergence.

OBSERVATION

Decode by applying, never by reducing under lambdas

Natural

```
value STEP BASE

STEP = lam tail. lam c.
  lam z. c tail
BASE = lam c. lam z. z
```

CBV builds a finite spine. Python counts its c-nodes up to alpha.

Boolean

```
value MARK_T MARK_F

MARK_T = lam p. lam q.
  lam r. p
MARK_F = … lam r. q
```

Three binders, so a numeral misused as a boolean is malformed, not a wrong answer.

Pair

```
FST value
SND value
```

Evaluate each projection as a term, then observe it by its kind, recursively for nested pairs.

The caller supplies the kind (NUM(0) = FALSE). Outcomes: **decoded**, **exhausted** or **malformed**, with a separate step count.

WALKTHROUGH

2 + 3 * 4, end to end

1	**Source AST**	`D0Eop2("+", D0Eint(2), D0Eop2("*", D0Eint(3), D0Eint(4)))`
2	**Compile**	ADD NUM(2) (MUL NUM(3) NUM(4)), expanded to a 194-character pure term
3	**Execute**	Value after **4 β-steps**: a lambda with the arithmetic still suspended inside
4	**Observe as nat**	value STEP BASE gives a spine of **14** nodes in **25 β-steps**
5	**Separate check**	The shared evaluator runs the *source* AST: 14. Decoded 14 = expected 14. **PASS**

```
$ python3.12 MySolution/demo.py --only "2 + 3"
```

RECURSIVE PROGRAM

fact 4 through Z: 24, by β-reduction alone

```
(fix fact(n). if n == 0 then 1 else n * fact (n - 1)) 4
```

1,544	471	218	24
characters of pure term	execution β-steps	observation β-steps	decoded = expected = source

fib 5 with fib 0 = 0, fib 1 = 1: 1,494 characters, 2,021 execution steps, 45 observation steps, decoded **5**.

Compiling LAMBDA · 21 · Budgets: 100,000 execution, 100,000 observation

RESULTS

All 14 required demonstrations match

Program	Result	Exec	Obs
2 + 3 * 4	14	4	25
2 − 5	0	87	2
if 2 <= 3 then 7 else 9	7	77	9
let x = 3 in (lam y. x + y) 4	7	4	13
fst (pair 2 true)	2	6	4
snd (pair 2 true)	true	6	2
(lam f. f 3) (lam x. x + 1)	4	4	10

Program	Result	Exec	Obs
true and false	false	3	2
false or true	true	3	2
if true then 1 else LOOP	1	3	3
(lam x. 0) LOOP	exhausted	100,000	not run
fact 4	24	471	218
fib 5	5	2,021	45
LOOP	exhausted	100,000	not run

6 more pass too: nested pairs, (fix f(f). f) 3, false and LOOP, not (3 != 3), encoding names as source names, let x = LOOP in 2.

DESIGN FOR TESTABILITY

Built so the claims can be checked

Seams

Budget and trace parameters on the evaluator. Its entry points can be swapped for failing mocks to prove compilation never evaluates.

An independent oracle

The source path never compiles, and the pipeline never applies source rules. Differential tests compare them.

Hazard-seeking generator

Random programs draw names from s, z, t, f, g, x, v, u_1, v_1 on purpose, so collisions happen constantly. Seeded, so failures reproduce.

Traceability

Tests cite REQUIREMENTS.md acceptance IDs, so each obligation points to code that checks it.

Determinism and immutability make **exact assertions** possible: the same term, answer and step count every run.

TESTING

137 unittest cases, one row per obligation

Obligation	Test files (count)	Representative cases
1 · Substitution, capture	target (15)	x := y in lam y. x gives lam y_1. y
2 · CBV order, budgets	reducer (18)	Budget 0 / 1 edges; nothing under lambdas
3 · Every constructor, operator	church (30), compiler (38)	pred 0; both comparison outcomes; truth tables
4 · Scope and names	compiler	let x = x in x; fix f(f); free s, z, t, f, x
5 · Divergence, laziness	compiler, source_eval (27)	false and LOOP skipped; pair 2 LOOP exhausts
6 · Demos, rejection	compiler	All 14 demos; −3, /, %, div in dead code
7 · Structure	compiler, modules (4)	Pure output; closed library; evaluator mocked

Seeded property tests (4)

250 random closed programs: decoded result = source result. 250 open: pure, FV kept, deterministic, unmutated.

```
python3.12 -m unittest discover -s
MySolution -t MySolution
```

137 tests, OK, about 5 s

CORRECTNESS

Why the translation preserves meaning

Claim. For a closed, well-typed program whose result is a natural, a boolean, or a finite pair of these, observing C[e] gives the same data as evaluating e, given enough steps.

Application

Both sides evaluate function, then argument, then substitute. C maps values to values and commutes with substitution.

Conditional

C[c] reaches TRUE or FALSE, which picks one thunk. I forces only that one: same branch, same divergence.

Let

(lam x. C[e2]) C[e1] evaluates C[e1] first, then substitutes it into the body, exactly the source let rule.

Recursion

Z G v unfolds G once per call, matching fix f(x). e applied to v, which steps to e[f := fix f(x). e][x := v].

Testing is evidence, not proof: hundreds of agreeing programs don’t cover every program. A proof needs induction on evaluation.

DESIGN RATIONALE

Decisions, alternatives and why

Decision	Instead of	Because
Visitor for source, match for target	Visitor everywhere	3 node types we own; match is shorter
Explicit-stack machine	Recursive evaluator	100,000-step runs; residual program on exhaustion
Combinators inlined	A let-bound prelude	Each term self-contained; closed pieces can’t capture
Thunks forced with I	Church if directly	CBV would run both branches
Fresh u_k, v_k per thunk	Reserved names	The spec forbids reserving source names
Substitution reducer	Environment machine	Matches the spec’s β-value rule; easy to trust
Three-binder markers	Matching TRUE / FALSE	Misuse shows as malformed, not a wrong answer

The pattern: **prefer what’s obviously correct**, then measure the cost (next slide).

LIMITATIONS

What it costs and what it leaves out

Unary numerals are slow

Cost grows with the value; SUB is about m·n steps. fib 5 already needs 2,021.

Substitution reducer

Each β-step rebuilds the body. An environment machine with sharing would be faster.

Large output

Combinators expand at every use: fact is 1,544 characters. Self-contained, not compact.

Closed programs run

Execution rejects open terms with OpenTermError. Compiling open terms works.

No type checking

Ill-typed programs compile. Observing them may report malformed or a meaningless value.

Kinds must be given

Decoders need nat, bool or a pair kind; functions can’t be decoded.

Optional extensions, outside the graded core and tested separately: a parser that inverts the pretty-printer, and a Flask web front end.

Thank you. Questions?

A compositional visitor compiles every constructor to pure terms.

Thunks and Z make call-by-value behave like the source.

One evaluator, separate budgets, 137 tests, 20 of 20 demos.

```
cd assigns/06
python3.12 -m unittest discover -s MySolution -t MySolution
python3.12 MySolution/demo.py
```